RateGuard: Rate-Limit and Quota Decision Service

Project Initiation Document (PID) | Version 1.0 | Status: Approved for build

1. Document Control

Item	Detail
Project name	RateGuard
Type	Backend service, individual project
Primary stack	Redis, PostgreSQL, one backend language of the builder's choice, Docker Compose
Target duration	4 weeks part-time (about 10 hours per week), MVP at end of week 2
Deliverables	Running service, test suite, benchmark report, README, demo script
Honesty rule	No performance figure, user count or business impact is claimed unless measured by the builder

2. Executive Summary

RateGuard is a standalone decision service that answers one question for an API provider: should this request from this client be allowed right now? It combines two controls that real APIs need together. A rate limit protects the system from short bursts using ephemeral counters in Redis. A quota enforces a billing-period allowance (for example calls per month) using durable usage records in PostgreSQL. The service exposes multiple algorithms (fixed window, sliding window counter, token bucket) behind one interface, per-plan policies, configurable behaviour when Redis fails, a shadow mode for safe rollout, and a benchmark harness that measures how the algorithms and the two quota strategies actually behave under concurrent load.

The project is deliberately not a claim to beat existing gateways. Its value is a correct, tested, measured implementation with documented engineering trade-offs.

3. Problem Statement

3.1 Context

Any API shared by many clients faces four recurring situations: one client sends a traffic spike, one client runs a buggy script that loops, one client abuses the service on purpose, and low-priority traffic competes with important traffic. Separately, commercial APIs sell plans with usage allowances that must be enforced exactly or nearly exactly.

3.2 Who experiences the problem

- API provider teams that run several instances of a service behind a load balancer.
- Platform teams that expose plans (free, standard, premium) with different limits.
- Developers integrating with APIs who need clear signals (how long to wait, how much is left).

3.3 Why simple approaches fail

1. **Per-process counters.** With three app instances, a limit of 100 per second becomes up to 300 per second, because each instance counts alone.
2. **Read-then-write logic in application code.** Two concurrent requests both read the same counter, both pass, both write. The limiter admits more than the limit. This is a race condition, not an edge case.
3. **Fixed windows only.** A client can send a full allowance at the end of one window and another full allowance at the start of the next, producing up to twice the intended rate across the boundary.
4. **Rate limit and quota treated as the same thing.** Rate counters can be lost on restart without harm. Billing usage cannot be lost casually. They need different storage and different guarantees.
5. **No defined failure behaviour.** If the limiter's store is down, the system either blocks everyone or lets everyone through, and nobody decided which on purpose.
6. **No safe rollout.** A new limit turned on directly can block legitimate traffic. There is no way to observe what would have been blocked first.

3.4 Problem statement

Provide a central, concurrency-correct service that decides allow or deny for each API request, enforcing short-window rate limits and billing-period quotas per client and plan, with explicit failure behaviour, safe rollout controls, and evidence (tests and benchmarks) of how its algorithms and design choices behave.

4. Objectives and Success Criteria

ID	Objective	Verifiable by
O1	Limits hold across multiple service instances	Test: 3 instances, concurrent load, total admitted within the algorithm's theoretical bound
O2	No over-admission from races	Race test: naive implementation fails, Lua implementation passes, same test
O3	Algorithm trade-offs are demonstrated, not asserted	Boundary-burst test and accuracy comparison against exact baseline
O4	Quota strategies compared	Exact (PostgreSQL) vs write-behind (Redis) test with measured over-admission and latency
O5	Failure behaviour is configurable and tested	Redis-down tests for fail-open and fail-closed
O6	Safe rollout supported	Shadow mode logs would-block decisions without enforcing
O7	Explainable in 60 to 90 seconds	Written interview answer in section 27

5. Scope

5.1 In scope

Decision API, three algorithms plus an exact baseline for testing, plan and policy management, quota tracking, hot configuration reload, fail-open and fail-closed modes, shadow mode, global bypass switch, usage reporting, metrics and structured logs, a small demo API with middleware, tests, benchmarks, documentation.

5.2 Out of scope

User interface, billing or payments, multi-region replication, Kubernetes, message brokers, machine-learning based throttling, IP reputation, authentication of end users (the service identifies clients by API key only).

5.3 Assumptions

- One Redis instance for the project. Redis Cluster considerations are documented, not built.
- Callers are trusted internal services (the demo middleware), authenticated by a service token.
- Quota period is a calendar month in UTC.
- API keys are high-entropy random strings issued by this service.

5.4 Constraints

Single developer, free local infrastructure, no paid APIs, no proprietary data.

6. Existing Solutions and Positioning

This section records what exists so the project's claims stay honest. Items marked (verify) come from general knowledge and must be checked before being cited in a report or interview.

Tool	What it is	Relevance
Stripe's published rate-limiter design	Four limiter types in production, token bucket, Redis, fail-open advice, dark launch, kill switch	Primary design reference for this project
Envoy ratelimit service (Lyft reference)	Go/gRPC service, config from disk, descriptors, Redis backend	Closest standalone equivalent; Kong Mesh reuses it with a fixed window algorithm
Kong rate-limiting plugin	Gateway plugin, counters in Redis or PostgreSQL	Gateway built-in (as described by a competitor's comparison, verify)

Tool	What it is	Relevance
nginx limit_req	Per-process limiting	Shows the per-process weakness (verify)
Upstash ratelimit library	Serverless-friendly library with fixed window, sliding window, token bucket, analytics, multi-region	Nearly the same algorithm set, as a library
redis-cell	Redis module exposing a throttle command	Shows an alternative to Lua scripts
Sentinel (Alibaba)	Local flow control component	Local vs central comparison
Gubernator, Bucket4j, Resilience4j, rate-limiter-flexible, AWS API Gateway usage plans, Cloudflare rate limiting rules	Other libraries, services and managed features	(verify)

Positioning

RateGuard does not try to replace these tools. It differs in three ways: it models rate limit and quota together in one policy; it exposes failure and rollout behaviour as first-class configuration; and it ships its own correctness tests and benchmark harness so every design choice is backed by measurement.

7. Solution Overview

1. A calling service (an API gateway or middleware) sends each incoming request's API key, route and cost to RateGuard.
2. RateGuard resolves the client and plan, finds the matching policy, and runs the rate algorithm atomically in Redis.
3. If rate passes, RateGuard checks the client's monthly quota.
4. RateGuard returns a decision with limits, remaining amounts and retry information.
5. The caller enforces the decision (HTTP 429 with Retry-After) or, in shadow mode, only records it.

6. Administrators change plans and policies through an admin API; changes are picked up without restart.

8. Stakeholders and Personas

Persona	Need
API provider operator	Safe rollout, kill switch, visibility into who is blocked and why
Platform engineer	Predictable behaviour when Redis or PostgreSQL fails
API consumer	Clear wait times and remaining allowance
Builder (you)	A project explainable and demonstrable end to end

9. Functional Requirements

ID	Requirement
FR-1	POST /v1/check accepts api_key, route, cost (default 1) and returns a decision
FR-2	Resolve api_key to client and plan using a stored hash; unknown or revoked keys are denied with a specific reason
FR-3	Resolve policy by route using specificity order: exact route, then longest prefix, then wildcard, then plan default
FR-4	Support algorithms: fixed_window, sliding_window (counter variant), token_bucket; sliding_window_log exists as exact test baseline
FR-5	Cost-weighted requests: a request may consume more than one unit

ID	Requirement
FR-6	Quota per plan per calendar month; two enforcement modes, exact and write_behind, selectable per plan
FR-7	Hot reload of plans and policies without restart
FR-8	Per-policy fail_mode: open or closed, applied when the store needed for the decision is unavailable
FR-9	Per-policy shadow flag: decision is computed and recorded, but the response always says allowed with shadow true
FR-10	Global bypass switch: allow everything, still record metrics
FR-11	Admin API for clients, API keys, plans, policies, quotas, bypass
FR-12	GET /v1/usage reports current period usage for a client
FR-13	Response carries limit, remaining, retry and reset information; demo middleware maps it to standard headers
FR-14	Metrics and structured logs for every decision
FR-15	POST /v1/refund returns consumed units when a later check stage denies the request

Optional enhancements (only after MVP is complete and tested)

- Concurrency limiter: limits in-flight requests per client using leased entries with expiry.
- Deny list for specific keys.
- Optional request_id on /v1/check so a retried check from the caller does not consume twice.

10. Non-Functional Requirements

Area	Requirement
Correctness	Concurrency tests must pass repeatedly, not once
Latency	Decision path measured and reported at p50 and p99; no number is promised before measurement
Availability behaviour	Explicit and tested for Redis down and PostgreSQL down
Security	Hashed keys, service token auth, input validation, no secrets in logs
Observability	Metrics, structured logs, health and readiness endpoints
Portability	Entire system runs with one Docker Compose command
Maintainability	Algorithm interface isolates new algorithms; schema managed by migrations
Testability	Injectable clock for pure logic; real Redis and PostgreSQL for integration tests

11. Terminology

- **Rate limit:** maximum requests in a short time span; counters are ephemeral.
- **Quota:** maximum usage in a billing period; usage is durable.
- **Cost:** units a request consumes.
- **Policy:** rule binding a plan and route to an algorithm and limits.
- **Fail-open:** allow when the limiter cannot decide.
- **Fail-closed:** deny when the limiter cannot decide.
- **Shadow mode:** compute decisions without enforcing them.
- **Degraded:** a decision made under a fail mode because a dependency was unavailable.

12. System Architecture

12.1 Logical view

```
  API consumer
      |
      v
  Demo API (middleware)  ---- POST /v1/check ---->  RateGuard instance(s)
      |                                                |        |
      | allowed? -> forward / 429                      |        |
                                                       v        v
                                                     Redis    PostgreSQL
                                                (rate counters, (plans,
policies,
                                                 quota counters) keys, quota
usage)
```

12.2 Components inside a RateGuard instance

Component	Responsibility
HTTP layer	Routing, validation, auth of caller, error mapping
Identity resolver	Hash api_key, look up client and plan, cache result briefly
Policy cache	In-memory snapshot of plans and policies; swapped atomically on reload
Config loader	Polls config_version, reloads snapshot when it changes
Algorithm engine	One interface, one implementation per algorithm, each backed by a Redis script
Quota manager	Exact mode (PostgreSQL) and write-behind mode (Redis counter plus flusher)
Flusher	Background task writing Redis quota totals to PostgreSQL
Decision composer	Orders checks, applies shadow, bypass and fail modes, builds response

Component	Responsibility
Metrics and logging	Counters, histograms, structured decision logs

12.3 Deployment view (local)

Docker Compose runs: Redis, PostgreSQL, three RateGuard instances, a simple round-robin proxy in front of them, the demo API, and a load-test runner. The three instances exist to prove limits hold globally.

12.4 Technology choices

Technology	Why
Redis	Fast shared counters; scripts execute atomically, which removes the read-then-write race
Lua scripts	Run the whole algorithm step as one atomic operation inside Redis
PostgreSQL	Durable plans, policies, keys and quota usage with constraints and transactions
Docker Compose	One-command reproducible environment
Load tool (k6 or similar)	Free, scriptable load generation for benchmarks
Test framework of chosen language	Unit and integration tests

No message broker, no Kubernetes, no service mesh: none is justified by the problem.

13. Request Workflow

13.1 Decision workflow (POST /v1/check)

1. Authenticate the caller with the service token. Reject if invalid.
2. Validate body: api_key present, route non-empty, cost positive integer within a maximum.
3. If global bypass is on: record metric, return allowed with reason bypass.

4. Hash api_key and resolve client and plan from cache or PostgreSQL. Unknown key: deny with unknown_key. Revoked: deny with key_revoked.
5. Resolve policy from the in-memory snapshot using specificity order.
6. Run rate algorithm: one Redis script call with a timeout budget.
    - Redis error or timeout: apply policy fail_mode (open: allow with degraded true; closed: deny with limiter_unavailable) and skip to step 9.
    - Denied: build rate_limited response with retry_after_ms and go to step 9.
7. If the plan has a quota: run quota check.
    - Exact mode: atomic conditional update in PostgreSQL.
    - Write-behind mode: atomic increment-and-compare in Redis.
    - Quota denied: call refund for the units consumed in step 6, build quota_exceeded response.
    - Dependency failure: apply the quota fail_mode.
8. Build success response including remaining rate and quota.
9. If policy shadow is true: record the real decision, then return allowed with shadow true.
10. Record metrics and structured log, return response.

13.2 Why quota is checked after rate

Rate checks are cheap and reject abusive bursts early, protecting PostgreSQL. The cost is a refund when quota later denies. The refund is best effort; if it fails the client loses a few rate tokens, which is the safe direction.

13.3 Configuration change workflow

1. Admin calls an admin endpoint to create or update a plan or policy.
2. The change and an increment of config_version commit in one PostgreSQL transaction.
3. Each instance polls config_version on a short interval (for example every few seconds), reloads when it changed, and swaps the in-memory snapshot atomically so in-flight requests see a consistent view.
4. Staleness window equals the poll interval; this is documented behaviour.

13.4 Quota write-behind workflow

1. Check does an atomic Redis increment on the monthly total key and compares to the limit.

2. If the key is missing (first use or Redis restart), the instance initialises it from PostgreSQL using set-if-not-exists, so a Redis loss does not reset usage to zero.
3. The flusher periodically writes totals to PostgreSQL with used = GREATEST(used, total). Because the total is monotonic, the flush is idempotent and safe to repeat after a crash.
4. Documented loss window: if Redis loses data between flushes, usage since the last flush is lost.

13.5 Failure workflows

Failure	Behaviour
Redis slow or down	Per-policy fail_mode; degraded flag; metric increments; readiness reports degraded
PostgreSQL down	Decisions continue from the cached snapshot; exact quota follows its fail_mode; admin API returns 503; flusher retries
Instance crash	Stateless; other instances continue; no state to recover except Redis and PostgreSQL
Config reload error	Keep last good snapshot, log error, retry
Refund failure	Log and count; do not fail the decision

14. Algorithms in Detail

All Redis scripts use the Redis server clock (TIME command), never the application server clock, so clock skew between instances cannot distort limits. Each script is atomic.

14.1 Fixed window

- State: one counter per client, route and window index (key includes the window number).
- Step: increment by cost, set expiry on first increment, deny if total exceeds limit.
- Strength: cheapest, one key. Weakness: boundary burst, up to about twice the limit across two adjacent windows.

14.2 Sliding window counter

- State: counters for the current and previous window.
- Estimate: previous_count multiplied by the fraction of the previous window still overlapping the sliding window, plus current_count.
- Strength: smooth, cheap. Weakness: approximation that assumes even distribution inside the previous window.

14.3 Token bucket

- State: tokens and last refill timestamp (a Redis hash).
- Step: refill tokens by elapsed time times rate, capped at capacity; if tokens are at least cost, subtract and allow; otherwise compute retry time from the missing tokens.
- Strength: allows controlled bursts up to capacity, steady average rate. Weakness: two values per key, burst parameter needs explanation.

```
-- KEYS[1]=bucket; ARGV[1]=capacity, ARGV[2]=refill per second, ARGV[3]=cost
local t = redis.call('TIME')
local now_ms = t[1] * 1000 + math.floor(t[2] / 1000)
local d = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(d[1]) or tonumber(ARGV[1])
local ts = tonumber(d[2]) or now_ms
tokens = math.min(tonumber(ARGV[1]), tokens + (now_ms - ts) *
tonumber(ARGV[2]) / 1000)
local allowed, retry_ms = 0, 0
if tokens >= tonumber(ARGV[3]) then
  tokens = tokens - tonumber(ARGV[3]); allowed = 1
else
  retry_ms = math.ceil((tonumber(ARGV[3]) - tokens) * 1000 /
tonumber(ARGV[2]))
end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now_ms)
redis.call('PEXPIRE', KEYS[1], math.ceil(tonumber(ARGV[1]) /
tonumber(ARGV[2]) * 2000))
return {allowed, math.floor(tokens), retry_ms}
```

The script must be verified by the project's own tests before it is trusted.

14.4 Sliding window log (test baseline only)

- State: a sorted set of request timestamps.

- Step: remove entries older than the window, count, add if under the limit.
- Exact, but memory grows with traffic; used as ground truth to measure the approximation error of the counter variant.

14.5 Refund

A small script that adds units back to a token bucket (capped at capacity) or decrements a window counter (not below zero).

14.6 Redis key design

Key pattern	Purpose
rl:{client_id}:route:alg:window	Rate state; the braces are a hash tag so related keys of one client share a cluster slot
q:{client_id}:period	Write-behind quota total

Every key has a TTL so unused clients do not accumulate memory.

15. Data Design

15.1 PostgreSQL schema

```
CREATE TABLE plans (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  quota_limit BIGINT,                  -- NULL means no quota
  quota_mode TEXT NOT NULL DEFAULT 'exact' CHECK (quota_mode IN
('exact','write_behind'))
);

CREATE TABLE clients (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  plan_id BIGINT NOT NULL REFERENCES plans(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE api_keys (
```

```
  id BIGSERIAL PRIMARY KEY,
  client_id BIGINT NOT NULL REFERENCES clients(id),
  key_hash BYTEA NOT NULL UNIQUE,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN
('active','revoked')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE policies (
  id BIGSERIAL PRIMARY KEY,
  plan_id BIGINT NOT NULL REFERENCES plans(id),
  route_pattern TEXT NOT NULL,        -- exact, prefix ending in *, or *
  algorithm TEXT NOT NULL CHECK (algorithm IN
('fixed_window','sliding_window','token_bucket')),
  limit_units BIGINT NOT NULL CHECK (limit_units > 0),
  window_seconds INT NOT NULL CHECK (window_seconds > 0),
  burst BIGINT,                       -- token bucket capacity
  fail_mode TEXT NOT NULL DEFAULT 'open' CHECK (fail_mode IN
('open','closed')),
  shadow BOOLEAN NOT NULL DEFAULT false,
  UNIQUE (plan_id, route_pattern)
);

CREATE TABLE quota_usage (
  client_id BIGINT NOT NULL REFERENCES clients(id),
  period_start DATE NOT NULL,
  used BIGINT NOT NULL DEFAULT 0 CHECK (used >= 0),
  PRIMARY KEY (client_id, period_start)
);

CREATE TABLE config_version (
  id INT PRIMARY KEY CHECK (id = 1),
  version BIGINT NOT NULL
);

CREATE TABLE settings (
  key TEXT PRIMARY KEY,               -- for example 'bypass'
  value TEXT NOT NULL
);
```

Exact quota statement (zero rows updated means quota exceeded; a row is inserted first if absent):

```
UPDATE quota_usage SET used = used + $1
WHERE client_id = $2 AND period_start = $3 AND used + $1 <= $4;
```

15.2 Data handling rules

- API keys are never stored in plain text; only a SHA-256 hash of the high-entropy key is stored, compared in constant time.
- Migrations are versioned and repeatable.
- Redis holds no data that cannot be rebuilt or tolerated lost, except write-behind quota totals, whose recovery rule is in 13.4.

16. API Specification

16.1 Decision endpoint

POST /v1/check (service token required)

Request:

```
{ "api_key": "rg_live_...", "route": "POST /v1/payments", "cost": 1 }
```

Response (HTTP 200 for every decision; HTTP errors are reserved for service or request problems):

```
{
  "allowed": true,
  "reason": "ok",
  "policy_id": 12,
  "limit": 100,
  "remaining": 87,
  "retry_after_ms": 0,
  "reset_ms": 1000,
  "quota": { "limit": 1000000, "used": 12345, "remaining": 987655 },
  "degraded": false,
  "shadow": false
}
```

Reason codes: ok, bypass, rate_limited, quota_exceeded, unknown_key, key_revoked, limiter_unavailable.

Why HTTP 200 with a decision body: a decision service separates a decision (deny) from a malfunction (500). The caller's middleware turns a deny into HTTP 429 with a Retry-After header.

16.2 Other endpoints

Method and path	Purpose
POST /v1/refund	Return units after a later stage denies (internal)
GET /v1/usage/{client_id}	Current period usage and plan limits
POST, GET, PUT, DELETE /admin/v1/plans	Plan management
POST, GET, PUT, DELETE /admin/v1/policies	Policy management
POST, GET /admin/v1/clients	Client management
POST /admin/v1/clients/{id}/api-keys	Issue key; secret shown once
POST /admin/v1/api-keys/{id}/revoke	Revoke key
PUT /admin/v1/bypass	Turn global bypass on or off
GET /healthz	Process is alive
GET /readyz	Dependencies reachable; reports degraded
GET /metrics	Metrics for scraping

16.3 Error model

All errors return a JSON body with code and message, and the right HTTP status: 400 invalid input, 401 or 403 auth failures, 404 not found, 409 conflicts such as duplicate policy, 422 semantic validation, 503 dependency failure on admin operations.

16.4 Headers produced by the demo middleware

Retry-After on denial (standard HTTP header). Rate-limit information headers follow the IETF draft for RateLimit fields; confirm its current status before relying on exact names, and consider the widely used X-RateLimit-* names as a documented alternative.

17. Security Design

Concern	Control
Caller authentication	Service token on /v1 endpoints; separate admin token on /admin endpoints
Key storage	Hash only; plain key shown once at creation
Input abuse	Strict validation, maximum cost, maximum route length, request size limit
Log hygiene	Never log api_key; log client_id and key id
Timing attacks	Constant-time hash comparison
Admin misuse	Admin operations logged with actor and change
Resource exhaustion	TTLs on all Redis keys; bounded caches

The limiter's own endpoints should also be protected from overload: the decision path has a timeout budget and bounded concurrency so a slow Redis cannot exhaust the service.

18. Observability

- Counters: decisions by reason, by policy, by algorithm; degraded decisions; refunds; reload successes and failures.
- Histograms: decision latency, Redis call latency, PostgreSQL call latency.
- Gauges: config version, cache size, flusher lag.
- Structured log per decision: request id, client id, policy id, reason, latency, degraded, shadow.
- Shadow report: how many requests would have been denied per policy, enabling safe rollout.

19. Testing Strategy

ID	Test	Expected result
T1	Unit tests of each algorithm with injected time	Matches hand-calculated sequences

ID	Test	Expected result
T2	Race demo: naive read-then-write limiter vs Lua limiter under many concurrent callers	Naive over-admits; Lua stays within bound
T3	Fixed window boundary burst	Admits up to about twice the limit across the boundary; sliding window does not
T4	Sliding counter accuracy vs exact log baseline over random traffic	Error rate measured and reported
T5	Token bucket burst and refill	Burst up to capacity, then steady rate
T6	Three instances behind the proxy	Global admitted total within bound
T7	Cost-weighted requests	Units consumed correctly
T8	Policy specificity	Exact beats prefix beats wildcard beats default
T9	Hot reload	Changed limit takes effect within poll interval; bad reload keeps old snapshot
T10	Exact quota under concurrency	Never exceeds limit
T11	Write-behind quota under concurrency and simulated Redis restart	Over-admission or lost-count window measured and documented
T12	Redis down, fail-open and fail-closed	Behaviour matches policy; degraded flag set; recovery clean
T13	PostgreSQL down	Cached config still serves decisions
T14	Shadow mode	Always allowed in response, real decision logged
T15	Bypass switch	Everything allowed, metrics still recorded
T16	Refund path	Quota denial returns rate units
T17	Security checks	Invalid token rejected, keys never logged, hash comparison used
T18	Repeat runs	Concurrency tests pass many times in a row (flakiness check)

20. Benchmark Plan

Measure and report only what is measured. Record machine specification, versions and parameters with every result.

Benchmark	Metrics
Decision latency per algorithm	p50, p95, p99, throughput
Redis calls per decision	Count per algorithm
Memory per key	Bytes per client for each algorithm, measured with 100,000 simulated clients
Accuracy	Approximation error of sliding counter vs exact log
Quota exact vs write-behind	Latency, throughput, over-admission or loss window
Scaling across instances	Throughput with one, two and three instances
Failure behaviour	Time to detect Redis failure, recovery time

Output: a table in the README, with the script that reproduces it.

21. Repository Structure

```
rateguard/
  README.md            overview, quick start, design decisions, benchmark
table
  docs/PID.md          this document
  docker-compose.yml
  migrations/
  src/
    api/               HTTP handlers and validation
    algorithms/        one file per algorithm plus scripts
    quota/             exact, write-behind, flusher
    config/            snapshot, loader
    identity/          key hashing and resolution
    observability/
  demo-api/            small API with the limiter middleware
  tests/
    unit/
    integration/
    concurrency/
  bench/               load scripts and results
```

22. Development Plan

Phase	Duration	Work	Deliverable and acceptance
1 Foundation	Week 1 first half	Repository, Docker Compose, migrations, key issue and hashing, admin plan and policy create	Compose starts everything; a key can be issued and resolved
2 First algorithm	Week 1 second half	/v1/check, token bucket script, Redis wiring, timeout budget	T1, T5, T7 pass; race demo T2 written
3 Other algorithms	Week 2	Fixed window, sliding counter, exact log baseline, shared interface, refund	T2, T3, T4 pass. **MVP complete**

Phase	Duration	Work	Deliverable and acceptance
4 Policies and config	Week 3 first half	Specificity resolution, snapshot, hot reload, shadow, bypass	T8, T9, T14, T15 pass
5 Quota	Week 3 second half	Exact and write-behind modes, flusher, rehydration, refund flow	T10, T11, T16 pass
6 Resilience and security	Week 4 first half	Fail modes, readiness, service token auth, validation, logging hygiene	T12, T13, T17 pass
7 Evidence and polish	Week 4 second half	Multi-instance test, benchmarks, demo API, README, demo rehearsal	T6, T18 pass; benchmark table committed

If time runs short, cut in this order: optional enhancements, write-behind flusher polish, demo API. Never cut tests T2, T3, T6, T12.

23. Risks and Mitigations

Risk	Likelihood	Mitigation
Project dismissed as a tutorial topic	High	Lead every explanation with the measured evidence: race demo, boundary demo, quota comparison
Lua script bugs	Medium	Unit tests with controlled time; compare against exact baseline
Flaky concurrency tests	Medium	Fixed seeds, repeat runs (T18), tolerance based on theory, not exact equality
Scope creep into a gateway	Medium	Section 5.2 out-of-scope list is binding
Benchmark numbers misread	Medium	Record environment; avoid comparing against other tools unless run on the same machine

Risk	Likelihood	Mitigation
Redis cluster misunderstanding	Low	Document hash tags and single-slot rule; do not build clustering
Over-claiming	Medium	Resume and interview statements limited to what is tested

24. Definition of Done

1. All MVP functional requirements FR-1 to FR-15 implemented.
2. Tests T1 to T18 pass, concurrency tests repeated successfully.
3. docker compose up starts the whole system with three instances.
4. README contains quick start, architecture, trade-off explanations and the benchmark table with reproduction steps.
5. Demo script runs end to end.
6. No claim in any document that is not backed by a test or measurement.

25. Demo Script (10 minutes)

1. Start the system; show three instances behind the proxy.
2. Show a plan with a token bucket policy; send a burst, show allowed then rate_limited with retry time.
3. Run the race demo: naive limiter over-admits, Lua limiter does not.
4. Run the boundary demo: fixed window vs sliding window.
5. Show the same limit holding across three instances.
6. Stop Redis: show fail-open policy allowing with degraded true and fail-closed policy denying; restart and show recovery.
7. Turn on shadow mode for a policy; show would-block counts without blocking.
8. Show quota exact vs write-behind results.
9. Show benchmark table.

26. Interview Preparation

- Why Lua? What breaks without it?

- Redis executes commands one at a time; why is a script still needed?
- Fixed window vs sliding window vs token bucket: when would you choose each?
- Why use the Redis clock instead of the application clock?
- Fail-open or fail-closed: who decides, and for which endpoints?
- What happens if Redis restarts? Which data is lost and what is the impact?
- Why is quota in PostgreSQL and rate state in Redis?
- Explain the write-behind loss window and how the monotonic counter makes the flush idempotent.
- Why check rate before quota? What does the refund cost?
- How would you handle one very large client (hot key)?
- What changes in Redis Cluster?
- Central service vs in-process limiter: latency against accuracy?
- How does hot reload stay consistent for in-flight requests?
- What is the difference between a rate limiter and a load shedder?
- How would you extend this to multiple regions?

27. Sixty to Ninety Second Explanation

APIs need to limit clients fairly, but per-instance counters stop working once you run several instances, and read-then-write logic races. RateGuard is a central decision service. Each request is checked by an atomic Lua script in Redis using one of three algorithms, then against a durable monthly quota in PostgreSQL. I measured the trade-offs: the naive limiter over-admits under concurrency while the scripted one does not, fixed windows allow boundary bursts, and write-behind quota is faster than exact quota but has a documented loss window. Each policy chooses fail-open or fail-closed for Redis outages, and shadow mode lets operators see what a new limit would block before enforcing it.

28. Resume Entry

Built a rate-limit and quota decision service ([your stack], Redis, PostgreSQL) with token bucket, sliding window and fixed window algorithms implemented as atomic Lua scripts, per-plan policies with hot reload, configurable fail-open and fail-closed behaviour, shadow mode, and durable monthly quotas; verified correctness with concurrent and multi-instance tests and benchmarked algorithm accuracy and latency.

Add numbers only after they are measured.

29. Future Enhancements

Concurrency limiter, atomic multi-policy evaluation in one script, deny lists, request_id deduplication, Redis Cluster support, per-region limits, adaptive load shedding based on service health signals, a small read-only dashboard.

30. Open Decisions

Decision	Default
Implementation language	The one the builder can debug confidently
Config poll interval	A few seconds
Maximum cost per request	Configurable, small default
Quota period	Calendar month, UTC
Header naming	Retry-After plus documented rate-limit header choice

31. References

- Stripe engineering blog, Scaling your API with rate limiters (design reference).
- Envoy ratelimit service repository (descriptor model, Redis backend).
- Upstash ratelimit documentation (algorithm set, library features).
- redis-cell project (Redis module approach).
- Redis documentation: scripting, TIME command, key expiry, Cluster hash tags.
- PostgreSQL documentation: transactions, constraints, UPDATE with conditions.
- IETF HTTP API working group draft on RateLimit header fields (check current status).